

Exercise Project X

Student Name: Máté Szabó

Course: Deep Learning

In this exercise project, I worked with a simple neural network that was built manually using Python, without using libraries like TensorFlow or PyTorch. The goal of the task was not to create a perfect model, but to understand how a neural network really works under the hood. By changing different parameters in the code and observing the results, I learned many new things about training, loss, and learning behavior. In this text, I describe my experiences based on five main experiment ideas.

1. Playing with the Data Generator

One of the first things I tried was changing the data that the neural network was trained on. I adjusted the data range and the number of data points. When I used a very small dataset, for example around 30 samples, the network learned very fast, but the results were not very stable. Sometimes the loss went close to zero quickly, but small changes in data caused big differences in performance.

When I increased the dataset size to around 300–500 samples, the training became slower, but the learning process was more stable. The loss decreased more smoothly, and the predictions were more consistent. This helped me understand that more data usually helps the model learn more reliably, even in a simple neural network.

2. Adjusting Starting Weights and Biases

Next, I experimented with the starting values of weights and biases. At first, the weights and biases had simple fixed values. When I slightly changed them, I noticed that the training process could change a lot. Some starting values helped the model converge faster, while others caused the loss to decrease very slowly.

I also tested what happens when multiple weights have the same starting value. In some cases, this made learning worse, because neurons behaved too similarly. When the starting values were more different, the network seemed to learn better. This made it clear to me why random initialization is usually used in real neural networks.

```
ORIGINAL WEIGHTS AND BIASES
w1: 0
w2: 0
w3: 0
w4: 0
w5: 0
w6: 0
b1: 0
b2: 0
b3: 0

#####
NEW WEIGHTS AND BIASES
w1: 0.0
w2: 0.0
w3: 0.0
w4: 0.0
w5: 0.0
w6: 0.0
b1: 0.0
b2: 0.0
b3: 0.04
```

3. Changing the Learning Rate

The learning rate had one of the biggest effects on training. When I used a very small learning rate (like 0.005), the loss decreased very slowly. The model was learning, but it needed many iterations to improve.

When I increased the learning rate to values like 0.1 or 0.2, the model learned much faster. However, when the learning rate was too high, the training sometimes became unstable and the loss started jumping up and down instead of going down. This showed me that the learning rate must be chosen carefully. It also helped me understand why extreme values can break the learning process in such a simple neural network.

4. Inspecting the Loss and Convergence

I paid close attention to the loss values and the loss plot during training. When the model converged, the loss stopped decreasing and stayed near a minimum value. In most cases, the network converged quite fast because both the data and the network structure were very simple.

However, in some experiments, the model had trouble converging, especially when the learning rate or starting weights were not good. This helped me understand that convergence is not guaranteed and depends on many factors, even in a basic neural network.

5. Understanding How a Neural Network Learns

After doing these experiments, I feel I understand better how a neural network actually learns. The network starts with random or fixed weights and biases, makes predictions, calculates the loss, and then uses gradient descent to adjust the weights in a direction that reduces the loss.

The loss function tells the model how wrong its predictions are, and gradient descent shows how to change the parameters step by step. After training, the network uses the learned weights and biases to make predictions on new data. Seeing all this happen manually in code made the learning process much clearer than just using a framework.

Final Thoughts

This exercise helped me realize how much work neural network libraries do behind the scenes. While it is possible to build a neural network from scratch, it is not very practical for larger or more complex problems. Still, this experiment was very useful for understanding the core ideas of neural networks, such as loss, learning rate, gradient descent, and convergence.

Overall, this project helped me learn not just theory, but also how small changes in code can have a big impact on performance. It gave me a much deeper understanding of how neural networks really work.